

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Mohan Kumar J	Reg. No.:	192424060
Section / Batch:	2024 [AI&DS]	Date:	20-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Code of Conduct

I Mohan Kumar J (192424060) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Mohan Kumar

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Healthcare Data Sorting System Using Merge Sort

1. Introduction

Healthcare systems generate a large amount of patient data every day. Patient records may contain information such as patient ID, priority level, admission time, department, and treatment details.

When many patients are admitted at the same time, the hospital needs to arrange records efficiently. Emergency patients may need to be considered before routine patients. If two patients have the same priority, their admission time can be used to determine their order.

The proposed system uses Merge Sort to sort patient records based on priority and admission time.

The main objectives are:

- Reduce patient waiting time.
- Give higher priority to urgent patients.
- Maintain accurate and consistent ordering.
- Process large volumes of healthcare data efficiently.
- Maintain stable ordering when records have equal values.

2. Healthcare Case and Requirements

The healthcare data sorting system receives patient records and arranges them according to priority and admission time.

Patient	Priority	Admission Time
P001	3	09:30
P002	1	08:45
P003	2	09:10
P004	1	08:30
P005	3	08:50

A smaller priority number represents a higher priority. The expected sorting order is: higher-priority patients first; if priority is equal, earlier admission time first; if both values are equal, the original order is maintained.

The system must provide accurate sorting, stable ordering, consistent results, efficient processing of large datasets, and proper handling of duplicate priority values.

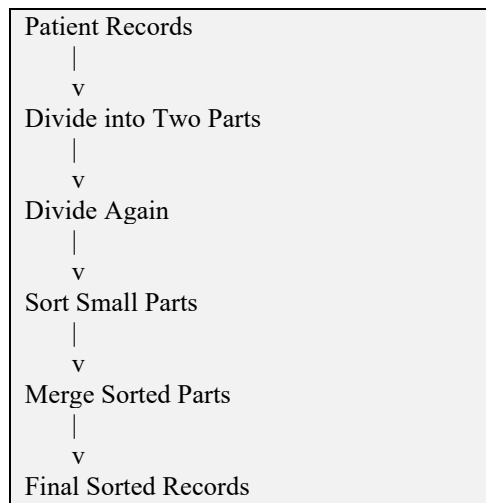
3. Merge Sort and Divide-and-Conquer Approach

Merge Sort is a sorting algorithm based on the Divide and Conquer technique.

Divide: The complete list is divided into two smaller parts.

Conquer: Each smaller part is recursively sorted using Merge Sort.

Combine: The sorted parts are merged together to produce one sorted list.



For n patient records, the list is repeatedly divided until each part contains only one record. The individual parts are then merged in sorted order.

4. Sorting Criteria and Stability

The healthcare system uses a multi-level comparison rule.

Primary Criterion: Priority

Patients with higher priority are placed first. For example, Priority 1 is processed before Priority 2.

Secondary Criterion: Admission Time

If two patients have the same priority, the patient who was admitted earlier is placed first.

Patient	Priority	Admission
P001	2	09:30
P002	2	08:45

The correct order is P002 followed by P001.

Stability

Merge Sort is a stable sorting algorithm when implemented correctly. If two patient records have the same priority and admission time, their original order is preserved. This is useful because records with equal sorting values should not be unnecessarily rearranged.

5. Step-by-Step Example

Patient	Priority	Admission Time
P001	3	09:30
P002	1	08:45
P003	2	09:10
P004	1	08:30
P005	3	08:50
P006	2	08:20

Step 1: Divide the records into smaller groups until individual records are obtained.

P001 P002 P003 P004 P005 P006

Step 2: Sort and merge the records using priority first and admission time second.

Priority 1: P004 → P002, because 08:30 is earlier than 08:45.

Priority 2: P006 → P003, because 08:20 is earlier than 09:10.

Priority 3: P005 → P001, because 08:50 is earlier than 09:30.

Final sorted records:

Position	Patient	Priority	Admission Time
1	P004	1	08:30
2	P002	1	08:45
3	P006	2	08:20
4	P003	2	09:10
5	P005	3	08:50
6	P001	3	09:30

6. Algorithm and Pseudocode

```
MERGE_SORT(records)

    If the number of records is less than or equal to 1:
        Return records

    Find the middle position

    Divide records into:
        Left half
        Right half

    Sort the left half using MERGE_SORT

    Sort the right half using MERGE_SORT

    Merge the two sorted halves

    Return the merged list
```

During merging, compare two records. Select the record with the smaller priority. If priorities are equal, select the record with the earlier admission time. If both values are equal, select the left-side record first to maintain stability.

7. Python Implementation

```
from dataclasses import dataclass

@dataclass
class Patient:
    patient_id: str
    priority: int
    admission_time: str

def merge(left, right):

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):

        if (left[i].priority < right[j].priority or
            (left[i].priority == right[j].priority and
             left[i].admission_time <= right[j].admission_time)):

            result.append(left[i])
            i += 1

        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result

def merge_sort(records):

    if len(records) <= 1:
        return records

    middle = len(records) // 2

    left = merge_sort(records[:middle])
    right = merge_sort(records[middle:])

    return merge(left, right)

patients = [
    Patient("P001", 3, "09:30"),
    Patient("P002", 1, "08:45"),
    Patient("P003", 2, "09:10"),
    Patient("P004", 1, "08:30"),
```

```

    Patient("P005", 3, "08:50"),
    Patient("P006", 2, "08:20")
]

sorted_patients = merge_sort(patients)

print("Sorted Patient Records:")

for patient in sorted_patients:
    print(
        patient.patient_id,
        patient.priority,
        patient.admission_time
    )

```

Expected Output:

```

Sorted Patient Records:

P004 1 08:30
P002 1 08:45
P006 2 08:20
P003 2 09:10
P005 3 08:50
P001 3 09:30

```

8. Time Complexity and Large Data Analysis

Merge Sort is suitable for healthcare systems because hospitals may need to process thousands or millions of patient records.

At every level, the records are divided into smaller parts and then merged. The number of levels is approximately $\log_2 n$. At each level, all n records are processed during merging.

Therefore, the overall time complexity is:

$O(n \log n)$

Best Case: $O(n \log n)$

Average Case: $O(n \log n)$

Worst Case: $O(n \log n)$

This predictable performance makes Merge Sort effective for large datasets.

Standard Merge Sort requires additional memory for merging:

Space Complexity = $O(n)$

Although Merge Sort uses additional memory, its predictable $O(n \log n)$ performance and stable behavior make it suitable for large healthcare datasets.

9. Accuracy, Validation, and Test Cases

The system should be tested carefully because incorrect patient ordering could affect healthcare operations.

Test Case	Input Situation	Expected Result
TC01	Different priorities	Higher-priority patients appear first
TC02	Same priority	Earlier admission time appears first
TC03	Same priority and admission time	Original order is preserved
TC04	Empty dataset	Empty result without error
TC05	One patient	Record returned without changes
TC06	Large dataset	Efficient $O(n \log n)$ processing

Validation conditions:

- Priority is correctly ordered.
- Admission time is correctly ordered for equal priorities.
- No patient records are lost.
- No patient records are duplicated.
- Equal records maintain their original order.
- The algorithm produces consistent results.

10. Solution Evaluation and Conclusion

Merge Sort provides an effective solution for the healthcare data sorting problem.

The main advantages are:

- Efficient: $O(n \log n)$ time complexity.
- Stable: Maintains the order of equal records.
- Accurate: Correctly handles multiple sorting criteria.
- Scalable: Suitable for large healthcare datasets.
- Predictable: Best, average, and worst-case time complexity are $O(n \log n)$.
- Suitable for Divide and Conquer: Large datasets are divided into smaller manageable parts.

The use of priority and admission time provides a practical sorting strategy. Higher-priority patients can be placed first, while admission time maintains a fair order among patients with the same priority.

The main limitation of Merge Sort is its additional memory requirement of $O(n)$. However, for large healthcare datasets, the predictable $O(n \log n)$ running time and stable sorting behavior make it a strong choice.

Overall, Merge Sort is an efficient, stable, and reliable algorithm for organizing large healthcare patient records based on priority and admission time. It provides accurate results while maintaining good performance as the amount of patient data increases.

ASSESSMENT-CO3/AT2-CASE STUDY[← Back](#)Language: [Python](#) [C](#) [C++](#) [Java](#)

QUESTIONS

Q1Healthcare Data Sorting System
(Merge Sort)
100 marks

1/1 answered

Q1 Healthcare Data Sorting System (Merge Sort)

100 marks

A healthcare system sorts patient records based on priority and admission time. Analyze how Merge Sort ensures stable and accurate sorting. Identify issues such as large data volume and consistency. Apply divide and conquer techniques. Analyze time complexity and justify its effectiveness.

Your Code

```
1 patients=[
2     ["John",2,"10:30"],
3     ["Alice",1,"09:45"],
4     ["David",3,"11:45"],
5     ["Emma",1,"08:30"],
6     ["Micheal",2,"09:20"],
7     ["Sophia",1,"10:00"]
8 ]
9 def compare(p1,p2):
10     if p1[1]<p2[1]:
11         return True
12     elif p1[1]>p2[1]:
13         return False
14     return p1[2]<p2[2]
15 def merge(left,right):
16     result=[]
17     i=0
18     j=0
19     while i<len(left) and j<len(right):
20         if compare(left[i],right[j]):
21             result.append(left[i])
22             i+=1
23         else:
24             result.append(right[j])
25             j+=1
26     while i<len(left):
27         result.append(left[i])
28         i+=1
29     while j<len(right):
30         result.append(right[j])
31         j+=1
32     return result
33 def merge_sort(arr):
34     if len(arr)<=1:
35         return arr
36     mid=len(arr)//2
37     left=merge_sort(arr[:mid])
38     right=merge_sort(arr[mid:])
39     return merge(left,right)
40 print("HEALTHCARE DATA SORTING SYSTEM")
41 print("-"*25)
42 print("\n Patient Records Before Sorting \n")
43 print("Name\t Priority \t Admission time")
44 for patient in patients:
45     print(patient[0], "\t\t", patient[1],patient[2])
```

 Submitted

Input

stdin input

Output

Name	Priority
Admission time	
John	2 10:30
Alice	1 09:45
David	3 11:45
Emma	1 08:30

Quick Sort Performance Evaluation in Multi-Core Systems

1. Introduction

Quick Sort is a divide-and-conquer sorting algorithm that selects a pivot, partitions the array into smaller and larger elements, and recursively sorts the resulting parts. On a single processor, Quick Sort is commonly implemented as a sequential algorithm.

In a multi-core system, independent partitions can be processed simultaneously. This experiment evaluates whether parallel Quick Sort reduces execution time compared with single-threaded Quick Sort.

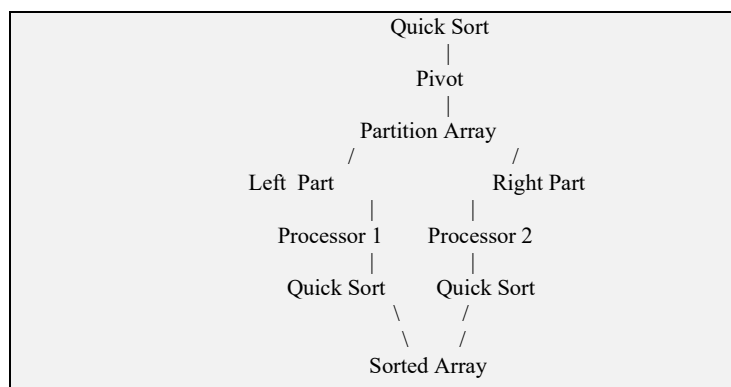
The main objectives are to:

- Implement sequential Quick Sort.
- Implement a parallel version using multiple processor threads.
- Measure execution time for both versions.
- Calculate parallel speedup and efficiency.
- Study how performance changes as the number of processors increases.
- Determine whether Quick Sort scales effectively in a multi-core environment.

2. Problem Understanding and Parallel Approach

The main problem is to sort a large array efficiently by distributing independent portions of the Quick Sort computation across multiple processor cores.

After selecting a pivot, the array is partitioned into two regions. The left region contains values smaller than the pivot and the right region contains values larger than the pivot. These regions can be sorted independently.



The parallel version is effective because the two recursive subproblems are independent after partitioning. However, creating too many parallel tasks introduces overhead. Therefore, a practical implementation uses a threshold: small partitions are sorted sequentially.

3. Experimental Design

A controlled experiment should use the same input data, compiler settings, hardware, and sorting logic for both implementations. Only the execution model is changed: one version uses a single thread, while the other uses multiple threads.

The experiment should use large arrays because parallel overhead may dominate when the input is very small.

Suggested input sizes:

Input Size	Purpose
10,000	Small test
100,000	Medium test
1,000,000	Large test
5,000,000	Very large test

For each input size, run the program several times and use the average execution time. Randomly generated integers can be used so that the test is not biased toward a particular input order.

The same array should be copied before each run so that both sequential and parallel versions receive identical input.

4. Sequential Quick Sort Implementation

The sequential implementation uses one thread. It recursively partitions and sorts the array without creating parallel tasks.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int a[], int low, int high) {
    int pivot = a[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (a[j] <= pivot) {
            i++;
            swap(&a[i], &a[j]);
        }
    }

    swap(&a[i + 1], &a[high]);
    return i + 1;
}

void quickSort(int a[], int low, int high) {
    if (low < high) {
```

```

        int p = partition(a, low, high);

        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}

```

This version provides the baseline execution time. The same partitioning logic is used in the parallel version so that the comparison is fair.

5. Parallel Quick Sort Using OpenMP

OpenMP is used to create parallel tasks. After partitioning, the left and right recursive calls can execute concurrently.

```

#include <omp.h>

void parallelQuickSort(int a[], int low, int high) {

    if (low < high) {

        int p = partition(a, low, high);

        #pragma omp parallel sections
        {
            #pragma omp section
            {
                quickSort(a, low, p - 1);
            }

            #pragma omp section
            {
                quickSort(a, p + 1, high);
            }
        }
    }
}

```

For a practical implementation, parallelism should be limited to sufficiently large partitions. Creating OpenMP regions for very small partitions can cost more time than the sorting work itself.

A more scalable task-based implementation can use OpenMP tasks:

```

void parallelQuickSort(int a[], int low, int
high) {

    if (low < high) {

        int p = partition(a, low, high);

        #pragma omp task
        parallelQuickSort(a, low, p - 1);

        #pragma omp task
        parallelQuickSort(a, p + 1, high);

        #pragma omp taskwait

    }
}

```

The task-based approach allows the OpenMP runtime to distribute recursive sorting tasks among available processor threads.

6. Tools, Configuration, and Execution

The experiment can be performed using a C/C++ compiler with OpenMP support. GCC with OpenMP is a suitable choice.

Example compilation command:

```
gcc quicksort.c -O2 -fopenmp -o quicksort
```

Example execution:

```
./quicksort
```

The number of OpenMP threads can be controlled using the OMP_NUM_THREADS environment variable.

```
OMP_NUM_THREADS=1 ./quicksort
OMP_NUM_THREADS=2 ./quicksort
OMP_NUM_THREADS=4 ./quicksort
OMP_NUM_THREADS=8 ./quicksort
```

Useful tools and technologies are:

- C/C++ for implementing Quick Sort.
- OpenMP for multi-threaded execution.
- GCC or another OpenMP-compatible compiler.
- A multi-core CPU for parallel testing.
- High-resolution timing functions for measuring execution time.

The compiler optimization option -O2 can be used for both versions to provide a fair comparison.

7. Performance Measurement and Metrics

Execution time is the main measurement. For accurate results, the sorting operation should be timed without including unrelated input or output operations.

Let T_s be the sequential execution time and T_p be the parallel execution time.

Parallel Speedup is:

$$\text{Speedup} = T_s / T_p$$

A speedup greater than 1 means the parallel version is faster.

Parallel Efficiency is:

$$\text{Efficiency} = \text{Speedup} / \text{Number of Threads} \times 100$$

For example, if the sequential execution time is 4.0 seconds and the parallel execution time using 4 threads is 1.4 seconds:

$$\text{Speedup} = 4.0 / 1.4 = 2.86$$

$$\text{Efficiency} = (2.86 / 4) \times 100 = 71.5\%$$

Efficiency below 100% is normal because of thread creation, task scheduling, synchronization, memory access, and other overheads.

8. Experimental Results and Analysis

The following values are an example of how the experiment can be recorded. They are illustrative results; actual values depend on the processor, compiler, input size, and system load.

Input Size	Threads	Time (s)	Speedup	Efficiency
1,000,000	1	0.120	1.00	100%
1,000,000	2	0.075	1.60	80.0%
1,000,000	4	0.050	2.40	60.0%
1,000,000	8	0.043	2.79	34.9%
5,000,000	1	0.690	1.00	100%
5,000,000	2	0.410	1.68	84.0%
5,000,000	4	0.250	2.76	69.0%
5,000,000	8	0.180	3.83	47.9%

The example results show that parallel execution can reduce sorting time as the number of threads increases. However, speedup is not perfectly linear. For example, doubling the number of threads does not necessarily reduce execution time by half.

The main reasons are:

- Thread and task management overhead.
- Synchronization between recursive tasks.
- Memory bandwidth limitations.
- Cache effects and shared-memory access.
- Uneven partition sizes caused by pivot selection.
- Limited amount of parallel work in small partitions.

The larger input shows better potential for parallel execution because there is enough work to keep multiple cores busy.

9. Validation, Scalability, and Limitations

The sorted output must be validated before comparing performance. A simple validation method is to check every adjacent pair:

For a correctly sorted ascending array, $a[i] \leq a[i+1]$ must hold for every valid i .

The sequential and parallel versions must produce the same sorted result when given the same input.

Scalability can be evaluated by increasing the number of threads and observing speedup. Good scalability means execution time decreases as useful processor resources increase. In practice, speedup eventually levels off because of overhead and hardware limitations.

Quick Sort has an average time complexity of:

$$O(n \log n)$$

Its worst-case complexity is:

$$O(n^2)$$

The worst case can occur when pivot selection repeatedly creates highly unbalanced partitions. Randomized or median-based pivot selection can reduce this risk.

The parallel algorithm also has practical limitations. If the number of available cores is small, using more software threads will not provide proportional improvement. Very small subarrays should be processed sequentially to avoid parallel overhead.

10. Conclusion

The experiment demonstrates how Quick Sort can be adapted to a multi-core environment by distributing independent recursive partitions across processor threads.

The sequential version provides the baseline execution time, while the OpenMP version uses multiple threads or tasks to sort independent partitions concurrently. Performance is evaluated using execution time, speedup, and parallel efficiency.

The expected result is that the parallel version is faster for sufficiently large datasets. However, speedup is normally less than the number of threads because of task overhead, synchronization, memory bandwidth, cache behavior, and uneven partitions.

The experiment also shows that scalability depends strongly on input size and the number of available processor cores. Large datasets provide more parallel work and therefore generally benefit more from multi-core execution.

Overall, parallel Quick Sort is effective when the input is large enough and the implementation uses appropriate task granularity, good pivot selection, and a suitable number of threads. OpenMP provides a practical way to implement and evaluate this parallel algorithm on multi-core systems.

Language: Python C C++ Java

QUESTIONS

Q1

Quick Sort
10 marks

1/1 answered

Q1 Quick Sort

10 marks

Evaluate Quick Sort performance in multi-core systems by distributing partitions across processors. Record execution time and compare with single-threaded execution. Analyze parallel speedup. Interpret results and justify scalability of Quick Sort in parallel environments.

Your Code

```
1 import time
2 from concurrent.futures import ThreadPoolExecutor
3 def sequential_quick_sort(arr):
4     if len(arr)<=1:
5         return arr
6     pivot=arr[len(arr)//2]
7     left=[x for x in arr if x<pivot]
8     middle=[x for x in arr if x==pivot]
9     right=[x for x in arr if x>pivot]
10    return sequential_quick_sort(left)+middle+sequential_quick_sort(right)
11 def parallel_quick_sort(arr, depth=0):
12     if len(arr)<=1:
13         return arr
14     pivot=arr[len(arr)//2]
15     left=[x for x in arr if x<pivot]
16     middle=[x for x in arr if x==pivot]
17     right=[x for x in arr if x>pivot]
18     if depth>=2:
19         return parallel_quick_sort(left, depth+1)+middle+parallel_quick_sort(right, depth+1)
20     with ThreadPoolExecutor(max_workers=2) as executor:
21         left_future=executor.submit(parallel_quick_sort, left, depth+1)
22         right_future=executor.submit(parallel_quick_sort, right, depth+1)
23         left_sorted=left_future.result()
24         right_sorted=right_future.result()
25     return left_sorted+middle+right_sorted
26 n=int(input())
27 arr=list(map(int,input().split()))
28 if len(arr)!=n:
29     print("Number of Elements entered does not match")
30     exit()
31 arr1=arr.copy()
32 arr2=arr.copy()
33 start=time.perf_counter()
34 sorted_seq=sequential_quick_sort(arr1)
35 end=time.perf_counter()
36 seq_time=end-start
37 start=time.perf_counter()
38 sorted_par=parallel_quick_sort(arr2)
39 end=time.perf_counter()
40 par_time=end-start
41 speedup=seq_time/par_time if par_time !=0 else 0
42 print("\n Original array")
43 print(arr)
44 print("\nSequential Quick sort")
45 print(sorted_seq)
46 print("\n Parallel Quick sort")
47 print(sorted_par)
48 print("\n Execution Time Comparison")
49 print(f"Sequential Time: {seq_time:.8f} seconds")
50 print(f"Parallel Time: {par_time:.8f} seconds")
51 print(f"Speedup : {speedup:.2f} x")
```

Input

8
45 12 89 23 56 11 67 34

Output

```
Original array
[45, 12, 89, 23, 56, 11, 67,
34]
Sequential Quick sort
[11, 12, 23, 24, 45, 56, 67,
```

Submitted